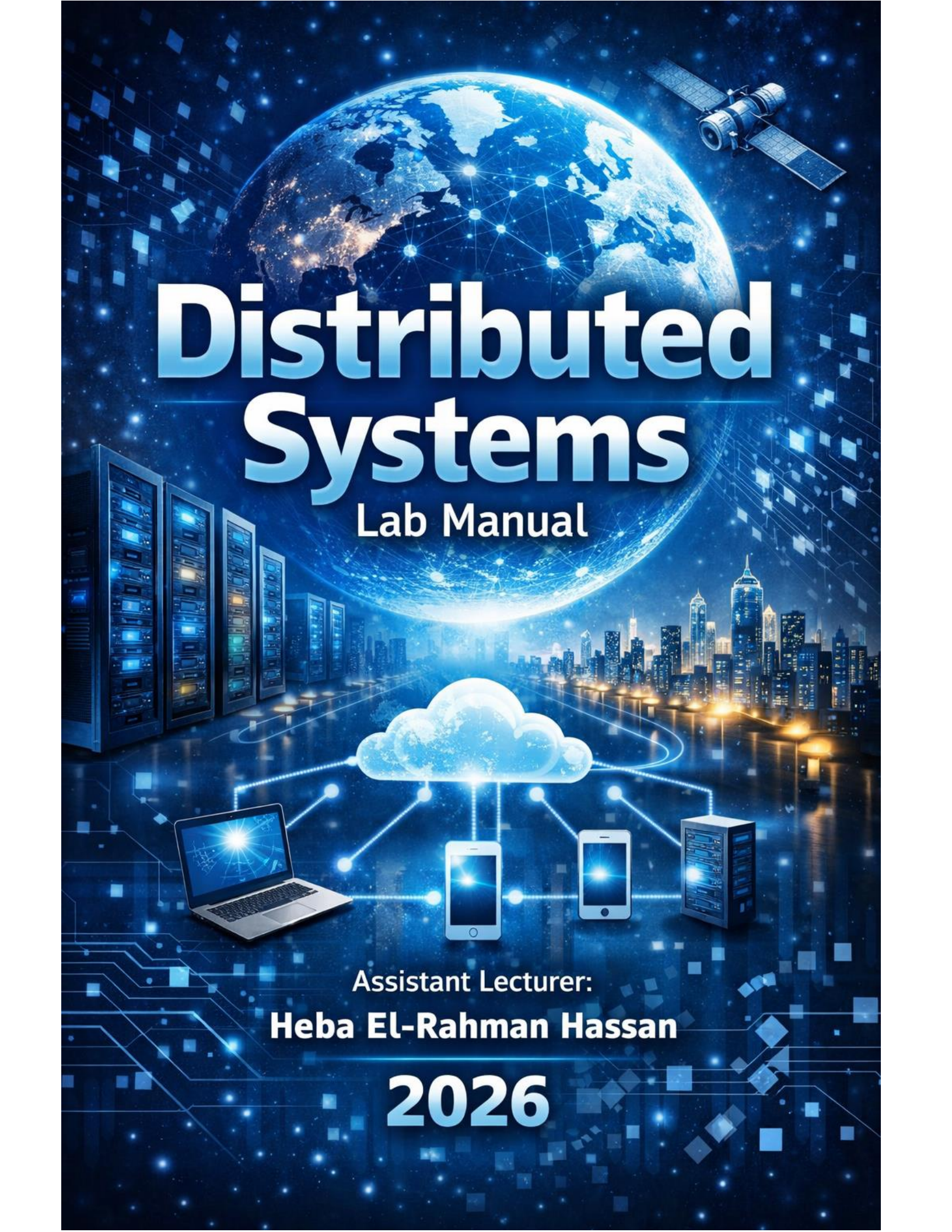




Distributed Systems

Lab Manual

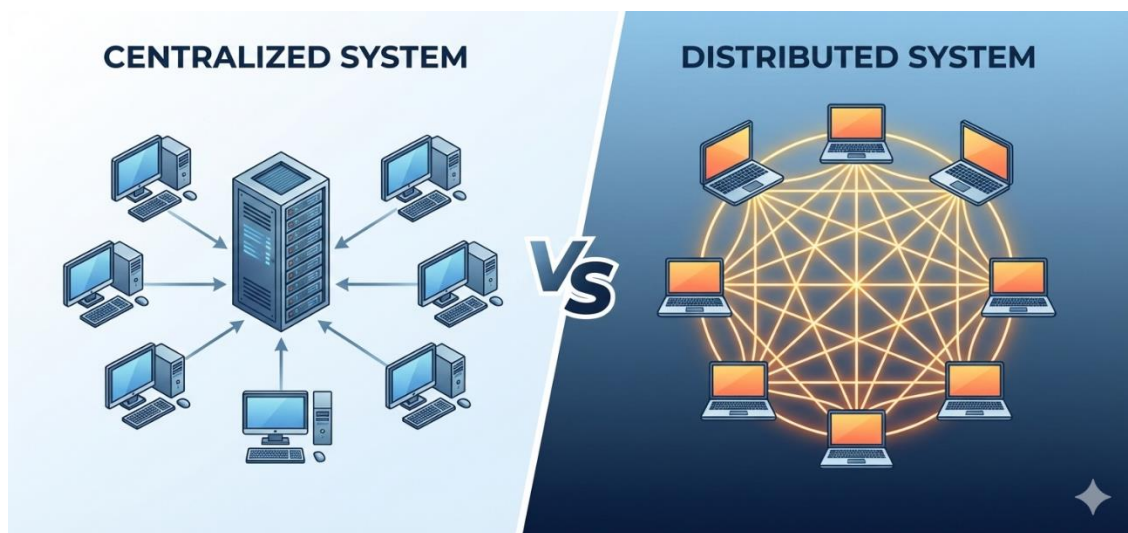
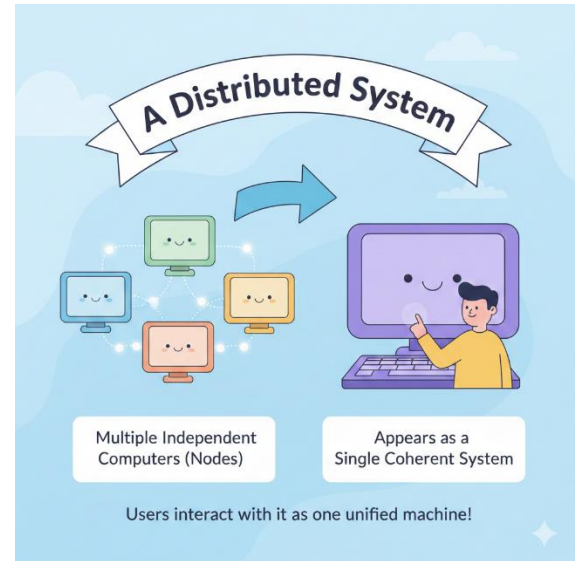
Assistant Lecturer:
Heba El-Rahman Hassan

2026

Chapter (1): Introduction to Distributed Systems

1.1 What Is a Distributed System?

- A **Distributed System** is a collection of independent computers that appear to users as a single coherent system
- Although multiple computers (also called *nodes*) are involved, users interact with the system as if it were one unified machine.
- These computers:
 - Work together
 - Communicate over a network
 - Coordinate their activities
 - Share resources, data, and tasks
 - Cooperate to achieve a common goal
- Unlike centralized systems — where all processing happens on one machine — distributed systems divide tasks across multiple machines to improve:
 - Performance
 - Reliability
 - Scalability



1.2 Why Do We Need Distributed Systems?

➤ Modern applications require:

- Large-scale data processing
- High availability
- Fault tolerance
- Geographic distribution
- Continuous service

➤ A single machine cannot efficiently handle these requirements. Distributed systems solve this problem by spreading computation across multiple machines.

➤ **Example:**

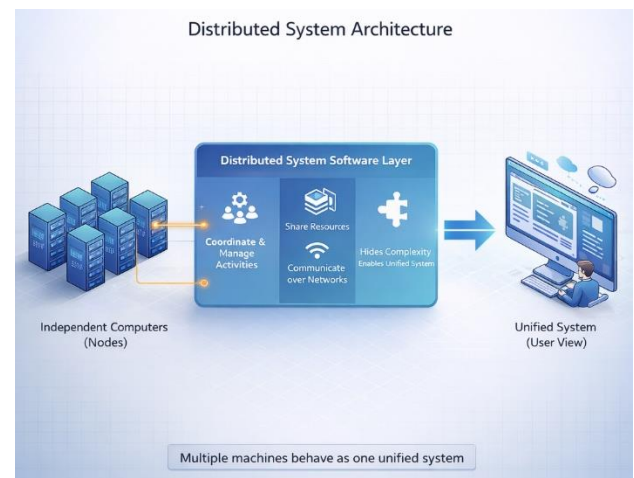
Google Search, Facebook, banking systems, cloud platforms, and online games are all examples of distributed systems.

1.3 Distributed System Software

➤ Distributed system software enables computers to:

- Coordinate their activities
- Share resources (hardware, software, and data)
- Communicate efficiently over networks

➤ This software layer hides complexity and makes multiple machines behave as one unified system.



1.4 Features of Distributed Systems

A distributed system typically possesses a set of structural and operational characteristics that enable multiple independent machines to function as a unified system.

These features can be categorized into **structural characteristics** and **system-level properties**.

1.4.1 Structural Characteristics

1.4.1.1 Multiple Machines

- Computations run on different machines (nodes).
- Each node performs part of the overall system task.

1.4.1.2 Communication Capability

- Machines communicate with each other through a network using standardized protocols such as TCP or UDP.
- Communication commonly occurs through **sockets**, which enable process-to-process interaction across machines.

1.4.1.3 Heterogeneity

- Machines in a distributed system may differ in:
 - Hardware configuration
 - Operating systems
 - Processing power
 - Programming languages
 - Network technologies
- The system must handle this diversity seamlessly.

1.4.1.4 Optional Centralized Server

- Some distributed systems rely on a central coordinating server (client–server model), while others operate in a fully decentralized manner (peer-to-peer systems).

1.4.1.5 Network-Based Communication

- All coordination and data exchange occur over a network.
- This makes reliable communication mechanisms essential for system correctness.

1.4.2 System-Level Properties

Beyond structural characteristics, distributed systems exhibit several important operational properties:

1.4.2.1 Scalability

- The system can grow in size — by adding more machines — without significant performance degradation.
- Scalability may be:
 - Horizontal (adding more nodes)
 - Vertical (upgrading machine resources)

1.4.2.2 Concurrency

- Multiple users and processes can access shared resources simultaneously.
- The system must manage:
 - Synchronization
 - Data consistency
 - Resource contention

1.4.2.3 Distributed Data

- Data is stored across multiple machines rather than in a single location.
- This improves:
 - Performance
 - Fault tolerance
 - Geographic distribution
- However, it introduces challenges such as:
 - Replication
 - Consistency
 - Data synchronization

1.4.2.4 High Availability

- The system remains operational even if some components fail.

- This is achieved through:
 - Replication
 - Redundancy
 - Failover mechanisms

1.4.2.5 Fault Tolerance

- The system continues functioning despite hardware or software failures.
- Mechanisms include:
 - Data replication
 - Checkpointing
 - Automatic recovery
 - Consensus algorithms

1.4.2.6 Security

- Since communication occurs over networks, distributed systems must ensure:
 - Authentication
 - Authorization
 - Data encryption
 - Secure communication channels
- Security is critical because multiple machines and users interact across potentially insecure networks.

1.4.2.7 Transparency

- Users perceive the distributed system as a single unified system.
- The system hides complexities such as:
 - Location of resources
 - Replication details
 - Failure recovery
 - Communication mechanisms
- Types of transparency include:
 - Location transparency
 - Replication transparency

- Failure transparency
- Concurrency transparency

1.4.2.8 Flexibility

- Distributed systems can adapt to:
 - New services
 - Changing workloads
 - New hardware
 - Evolving technologies
- Modern architectures such as microservices enhance flexibility.

Integrated View

A distributed system is not defined solely by multiple machines connected through a network. Its true strength lies in its ability to provide:

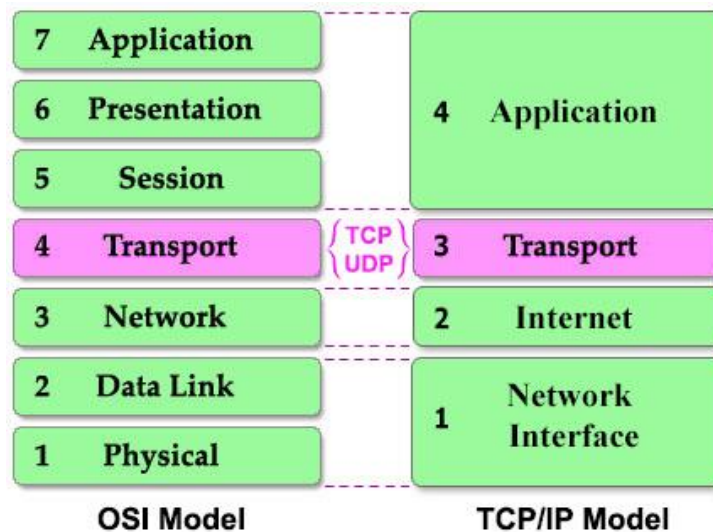
- Scalability
- Concurrency
- Distributed data management
- High availability
- Fault tolerance
- Security
- Transparency
- Flexibility

Together, these properties allow distributed systems to support large-scale, reliable, and modern computing environments.



1.5 Overview of Network Communication

- Distributed systems rely on network communication.
- The two main transport protocols used are:
 - **TCP (Transmission Control Protocol)**
 - **UDP (User Datagram Protocol)**
- These protocols operate at the Transport Layer of the TCP/IP model.



1.6 TCP vs UDP

TCP (Transmission Control Protocol)

TCP is:

- Connection-oriented
- Reliable
- Ordered
- Error-checked

It guarantees that:

- Data arrives
- Data arrives in order
- No duplication occurs

TCP is used in:

- Web browsing (HTTP/HTTPS)
- Email
- File transfers
- Banking systems

UDP (User Datagram Protocol)

UDP is:

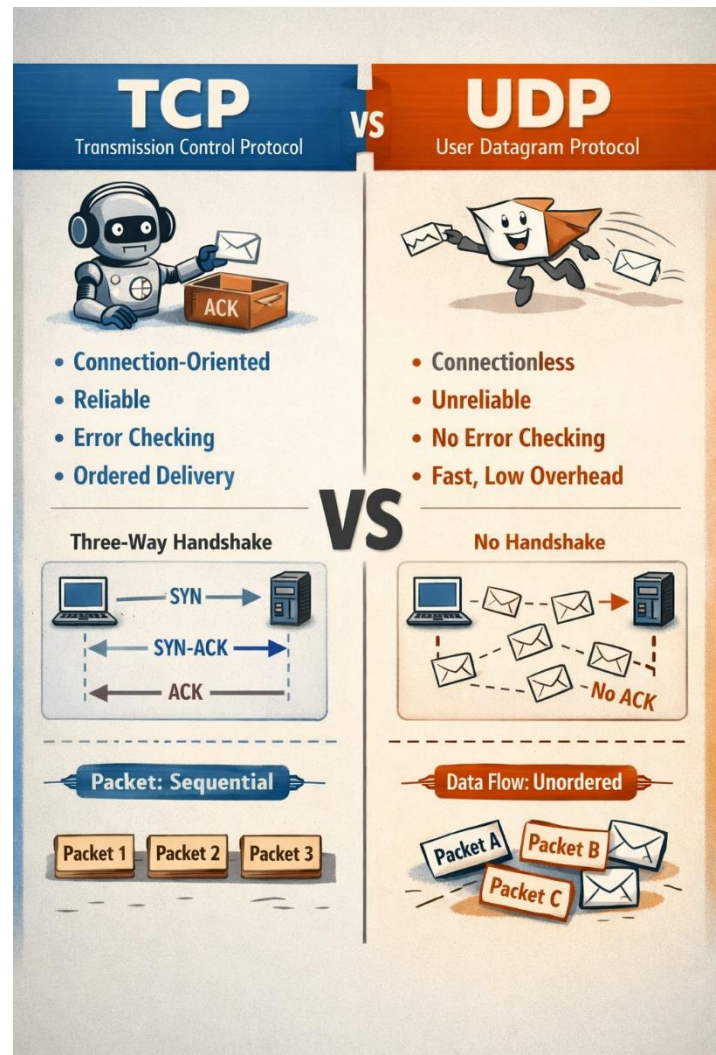
- Connectionless
- Faster
- Lightweight
- No delivery guarantee

It does not guarantee:

- Delivery
- Order
- Error recovery

UDP is used in:

- Video streaming
- Online gaming
- VoIP
- Real-time applications



1.7 TCP 3-Way Handshake

Before sending actual data, TCP establishes a reliable connection using a process called the **3-Way Handshake**

This mechanism ensures that both client and server are ready for communication.

Step 1: SYN

The client sends a synchronization (SYN) packet to the server.

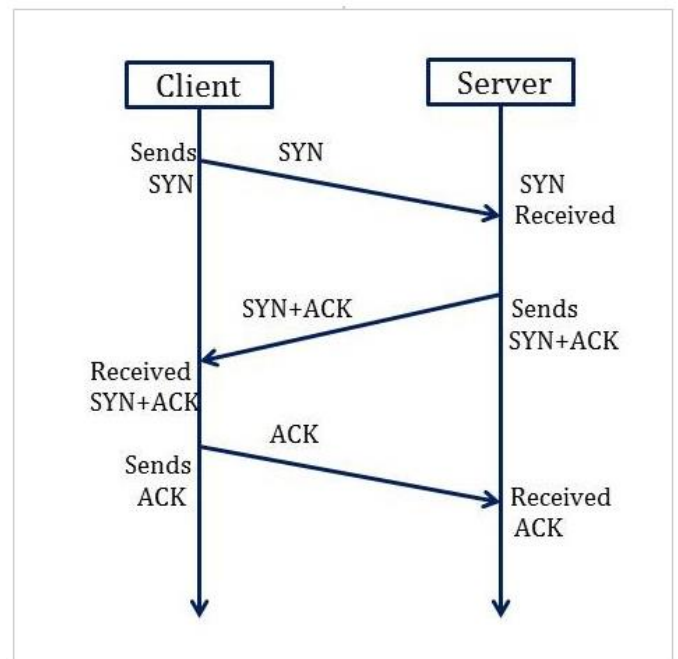
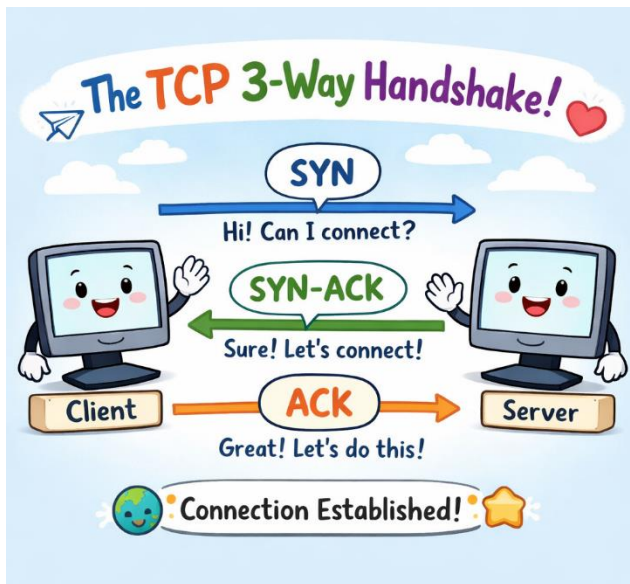
Step 2: SYN-ACK

The server responds with a synchronization-acknowledgment (SYN-ACK).

Step 3: ACK

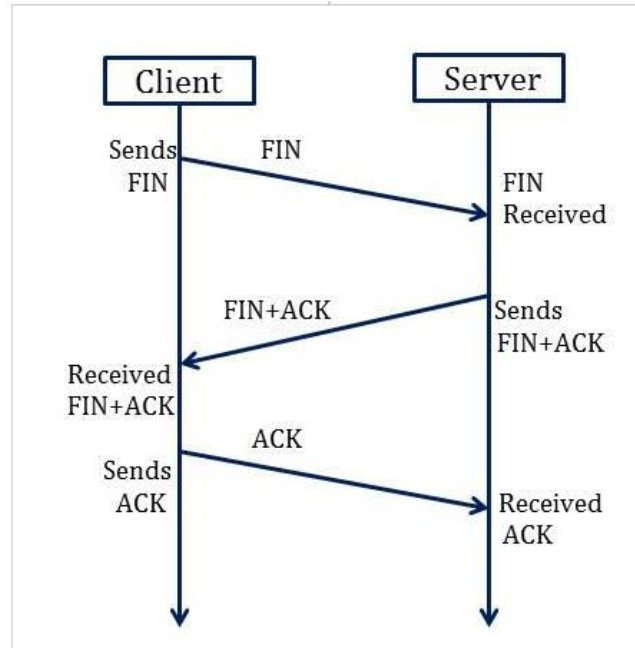
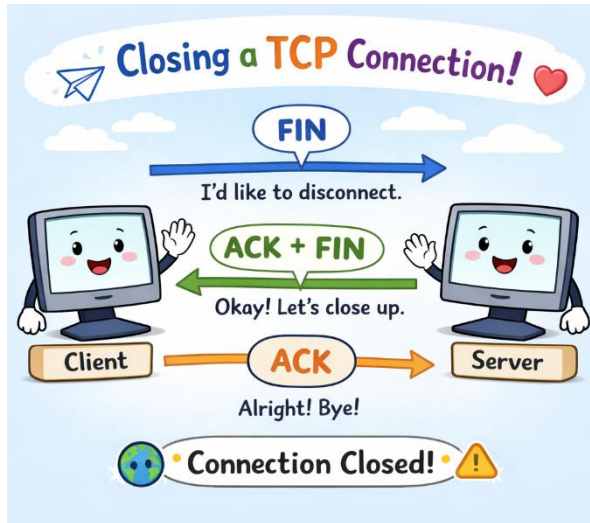
The client sends an acknowledgment (ACK).

After these three steps, the connection is established.



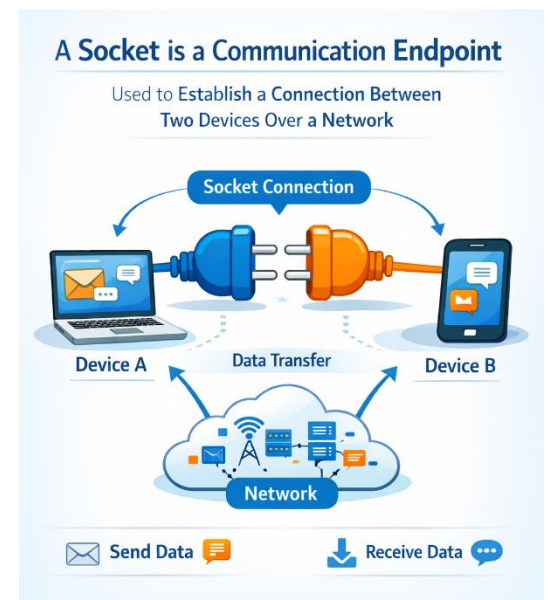
Closing a TCP Connection

Closing also involves a controlled multi-step process to ensure both sides terminate safely.



1.8 What Is a Socket?

- A **Socket** is a communication endpoint used to establish a connection between two devices over a network.
- Sockets are fundamental in distributed systems.
- They allow communication between:
 - Two different processes
 - On the same machine
 - Or on different machines
- Socket APIs are the standard interface for TCP/IP communication.



1.9 Socket Addressing

➤ To connect to a remote host, two pieces of information are required:

- **IP Address** → Identifies the machine
- **Port Number** → Identifies the service

➤ **Port Range:**

0 → 65,535

Ports 0 → 1,024 are reserved for well-known services:

Service	Port
HTTP	80
FTP	21
Telnet	23
SMTP	25

➤ A socket is defined as:

Socket = IP Address + Port Number

1.10 Traditional Socket Programming

➤ Sockets act as a layer between programs and the network card

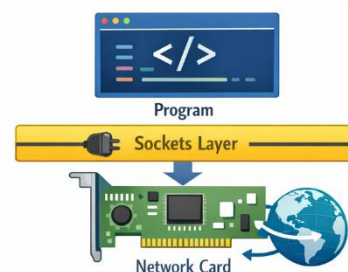
➤ Traditional socket programming involves:

- Low-level communication
- Manual data handling
- Byte-level transmission

➤ Important limitations:

- Only simple data types are transmitted (int, float, string)
- No direct method calls
- No automatic object transfer

Sockets: Interface Between Program & Network Card



Traditional Socket Programming



Low-Level Communication



Manual Data Handling



Byte-Level Transmission

Key Limitations



Simple Data Types Only
(int, float, string)



No Direct Method Calls



No Automatic Object Transfer

Challenges in Traditional Socket Programming

- Developers must manually handle:
 - Thread management
 - Control data (e.g., login, logout, authentication)
 - Business data (e.g., chat messages, file transfers)
 - Converting data to and from byte streams
- This complexity is one reason why higher-level distributed technologies (like RPC, RMI, REST APIs, gRPC) were later developed.

1.11 Server Socket vs Client Socket

Server Socket

- Listens for incoming connections
- Waits for client requests
- Accepts connection requests

Client Socket

- Initiates connection to a server
- Sends requests
- Receives responses

In summary:

- A **Server Socket** waits and accepts.
- A **Client Socket** connects and communicates.



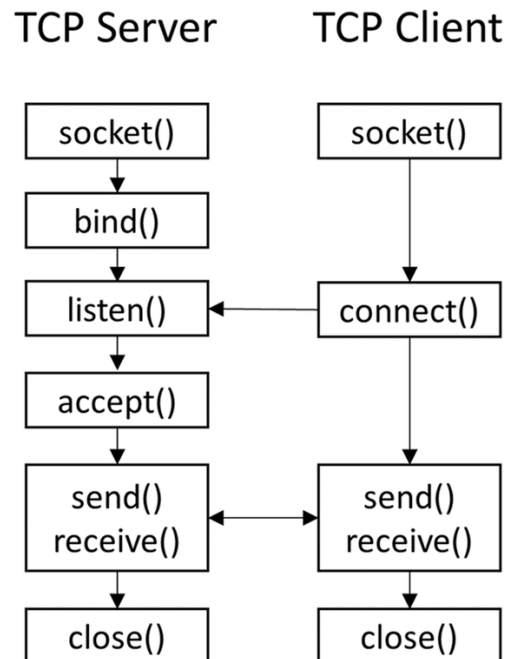
1.12 Basic Client/Server Communication

The client/server model is the foundation of distributed systems.

Basic Flow:

1. Server starts and listens on a port.
2. Client connects to server IP + port.
3. TCP 3-way handshake occurs.

4. Data exchange begins.
5. Communication continues until connection closes.



This model forms the basis of:

- Web applications
- Email systems
- Database systems
- Cloud services

Assignment 1

1. A distributed system is best defined as:

- A) A single powerful computer
 - B) A group of independent computers appearing as a single system
 - C) A centralized database system
 - D) A local desktop application
-

2. Which of the following is NOT a characteristic of distributed systems?

- A) Multiple machines
 - B) Network communication
 - C) Single point of failure
 - D) Resource sharing
-

3. Scalability in distributed systems mainly refers to:

- A) Increasing system security
 - B) Adding more machines to handle growth
 - C) Reducing system cost
 - D) Limiting user access
-

4. Horizontal scalability means:

- A) Increasing CPU power of one machine
 - B) Adding more machines to the system
 - C) Increasing RAM only
 - D) Decreasing system load
-

5. Which protocol guarantees reliable data delivery?

- A) UDP
 - B) FTP
-

- C) TCP
 - D) IP
-

6. UDP is preferred in which scenario?

- A) Online banking transactions
 - B) File transfer
 - C) Real-time video streaming
 - D) Email transmission
-

7. The TCP 3-way handshake includes:

- A) SYN, FIN, ACK
 - B) SYN, SYN-ACK, ACK
 - C) ACK, ACK, ACK
 - D) FIN, SYN, ACK
-

8. A socket is defined as:

- A) A hardware port
 - B) IP address only
 - C) Port number only
 - D) IP address + Port number
-

9. Which port range is reserved for well-known services?

- A) 1024–65535
 - B) 0–1023
 - C) 2000–3000
 - D) 3000–4000
-

10. Which of the following is a well-known HTTP port?

- A) 21
 - B) 25
-

- C) 80
 - D) 23
-

11. Concurrency in distributed systems means:

- A) Only one user can access the system
 - B) Multiple processes execute simultaneously
 - C) Data is stored in one location
 - D) No communication is required
-

12. Distributed data means:

- A) Data stored on a single server
 - B) Data stored across multiple machines
 - C) Data stored only in memory
 - D) Data stored offline
-

13. Fault tolerance ensures that:

- A) No errors ever occur
 - B) The system stops during failure
 - C) The system continues working despite failures
 - D) Users are blocked during failure
-

14. High availability means:

- A) The system is always expensive
 - B) The system remains operational most of the time
 - C) The system has no security
 - D) The system uses only TCP
-

15. Transparency in distributed systems hides:

- A) User passwords
 - B) Server hardware brand
-

- C) The complexity of distribution from users
 - D) Network cables
-

16. Heterogeneity refers to:

- A) Identical machines only
 - B) Different hardware and software environments
 - C) Same operating system everywhere
 - D) Same programming language
-

17. A server socket is responsible for:

- A) Initiating connection
 - B) Listening for incoming connections
 - C) Sending email
 - D) Encrypting passwords
-

18. A client socket is responsible for:

- A) Accepting connections
 - B) Listening continuously
 - C) Initiating connection to a server
 - D) Managing database replication
-

19. Which feature improves system reliability by duplicating data?

- A) Concurrency
 - B) Replication
 - C) Heterogeneity
 - D) Port addressing
-

20. Which of the following is a major security requirement in distributed systems?

- A) Data encryption
- B) Reducing scalability
- C) Removing authentication
- D) Disabling communication

Chapter (2): Socket Programming and One-to-One Client/Server Applications

2.1 Introduction

In chapter 1, we studied the theoretical foundations of distributed systems and their communication mechanisms.

In this chapter, we move from theory to practice by implementing **basic client/server communication using Java sockets**.

This chapter explains:

- The `ServerSocket` class
- The `Socket` class
- The `InetAddress` class
- How to build a simple one-to-one communication system
- How to design a basic chat application
- Best practices in exception handling

2.2 Java Networking Architecture

- Java provides built-in support for TCP/IP networking through the `java.net` package.
- Three core classes are used in basic socket programming:

1. `ServerSocket`
2. `Socket`
3. `InetAddress`



2.3 The ServerSocket Class

- The `ServerSocket` class is used on the **server side** to listen for incoming client connections.

- **Common Constructors**

```
ServerSocket(int port)  
ServerSocket(int port, int maxConnections)
```

- `port` → Port number where the server listens.
- `maxConnections` → Maximum queued client connections.

- **Common Methods**

```
Socket accept()  
void close()
```

- `accept()` → Waits for a client and returns a `Socket` object.
- `close()` → Closes the server socket.

2.4 The Socket Class

- The `Socket` class represents the communication endpoint between client and server.

- **Common Constructors**

```
Socket(String address, int port)  
Socket(InetAddress address, int port)
```

- `address` → Server IP address
- `port` → Server port number

- **Common Methods**

```
InputStream getInputStream()  
OutputStream getOutputStream()
```

These methods allow data exchange between client and server.

2.5 The InetAddress Class

- The `InetAddress` class represents IP addresses.
- It has **no public constructor**
- **Common Static Methods**

```
InetAddress.getByName(String host)
InetAddress.getAllByName(String host)
InetAddress.getLocalHost()
```

- **Other Useful Methods**

```
getHostName()
getHostAddress()
getAddress()
```

- This class helps resolve domain names and retrieve local or remote IP addresses.
-

2.6 Simple One-to-One Client/Server Application

A simple example where:

- The client sends a message.
- The server receives it.
- The server responds.
- The client prints the response.

2.6.1 Server Code Example

Important: `DataInputStream.readLine()` is deprecated.
We replace it with `BufferedReader`.

```
import java.io.*;
import java.net.*;

public class Server {

    public static void main(String[] args) {

        try (ServerSocket serverSocket = new ServerSocket(5005)) {

            System.out.println("Server is running...");
            Socket socket = serverSocket.accept();
            System.out.println("Client connected.");

            BufferedReader reader = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));

            PrintWriter writer = new PrintWriter(
                socket.getOutputStream(), true);

            String message = reader.readLine();
            System.out.println("Client says: " + message);

            writer.println("Data Received");

            socket.close();

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

2.6.2 Client Example

```
import java.io.*;
import java.net.*;

public class Client {

    public static void main(String[] args) {

        try (Socket socket = new Socket("127.0.0.1", 5005)) {

            BufferedReader reader = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));

            PrintWriter writer = new PrintWriter(
                socket.getOutputStream(), true);

            writer.println("Hello Server");

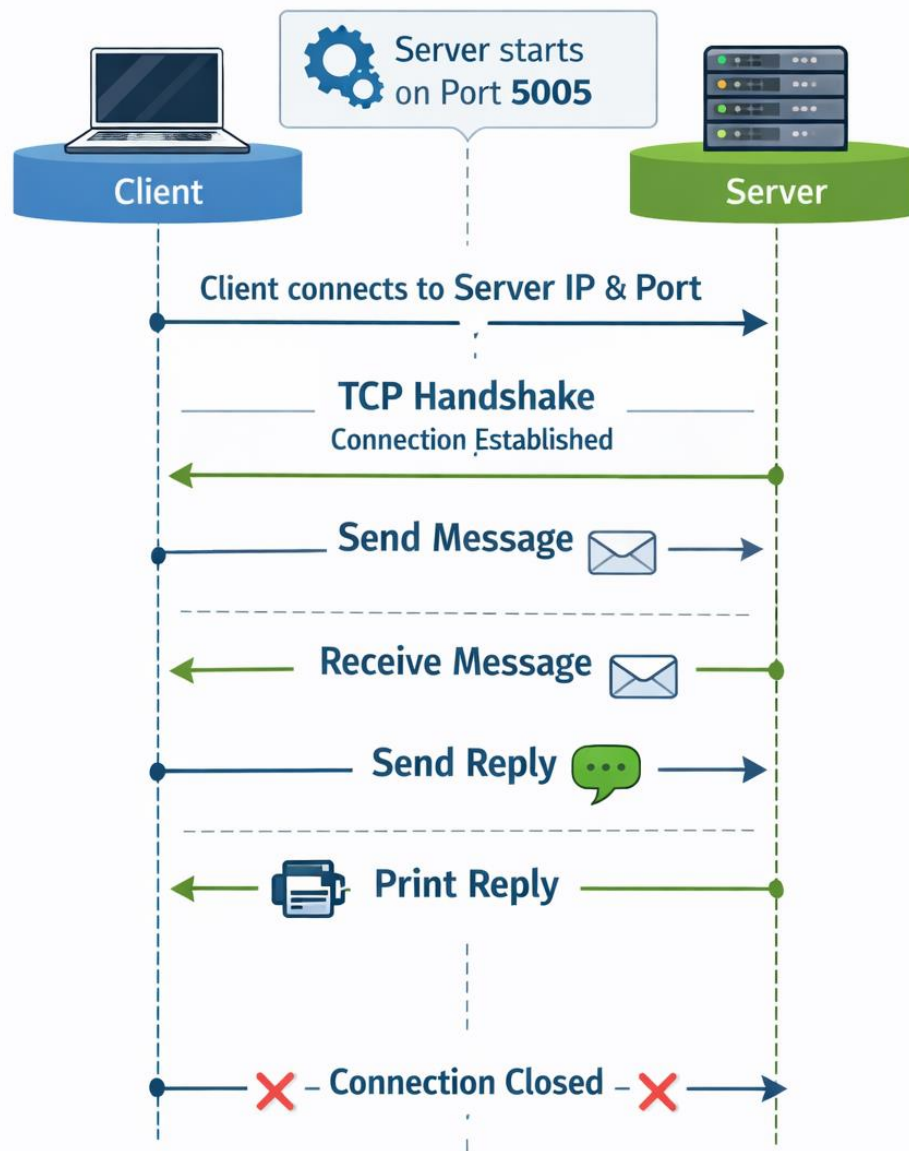
            String response = reader.readLine();
            System.out.println("Server says: " + response);

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

2.6.3 Communication Flow:

1. Server starts and listens on port 5005.
2. Client connects to server IP + port.
3. TCP handshake occurs automatically.
4. Client sends message.
5. Server receives message.
6. Server replies.
7. Client prints reply.
8. Connection closes.

Client-Server Communication Sequence



2.7 Chapter Summary

In this chapter, we learned:

- How Java supports TCP communication
- How `ServerSocket` listens for connections
- How `Socket` establishes communication
- How to build a simple client/server program
- How to implement a basic one-to-one chat system

This chapter bridges theory and implementation in distributed systems.

Assignment (2.1): One-to-One Chat Application

1. Write the simple client/server application.
2. Construct a GUI client-side chat application.
3. Handle all exceptions properly.
4. Replace deprecated methods.
5. Run the application on two real devices using real IP address (not loopback).

Tips to solve the assignment

➤ Running on Two Devices

→ Instead of:

127.0.0.1

Use:

192.168.x.x

→ To find your IP:

- Windows → ipconfig
- Linux/Mac → ifconfig

→ Both devices must be on the same network.

➤ Exception Handling Best Practices

Students must:

- Use try-with-resources
- Catch specific exceptions
- Always close sockets
- Avoid deprecated methods

This ensures robust distributed applications.

Assignment (2.2) – MCQ

1. The `accept ()` method belongs to:
A) Socket
B) ServerSocket
C) InetAddress
D) PrintWriter

2. Which class is used to resolve IP addresses?
A) Socket
B) ServerSocket
C) InetAddress
D) File

3. Which port range is reserved?
A) 0–1023
B) 1024–2000
C) 3000–4000
D) 4000–5000

4. TCP is:
A) Connectionless
B) Unreliable
C) Connection-oriented
D) Stateless

5. To send text data in Java sockets, we commonly use:
A) FileReader
B) BufferedReader
C) Scanner
D) System.in

6. `getInputStream()` returns:
A) Port number
B) InputStream

- C) OutputStream
- D) Socket

7. The client initiates connection using:
 - A) accept()
 - B) new ServerSocket()
 - C) new Socket()
 - D) getLocalHost()
8. In two-device testing, we must use:
 - A) 127.0.0.1
 - B) Real local IP address
 - C) Port 80
 - D) DNS only
9. The try-with-resources statement ensures:
 - A) Faster execution
 - B) Automatic resource closing
 - C) Better UI
 - D) Multi-threading
10. A chat system requires:
 - A) UDP only
 - B) TCP connection
 - C) No protocol
 - D) Only GUI

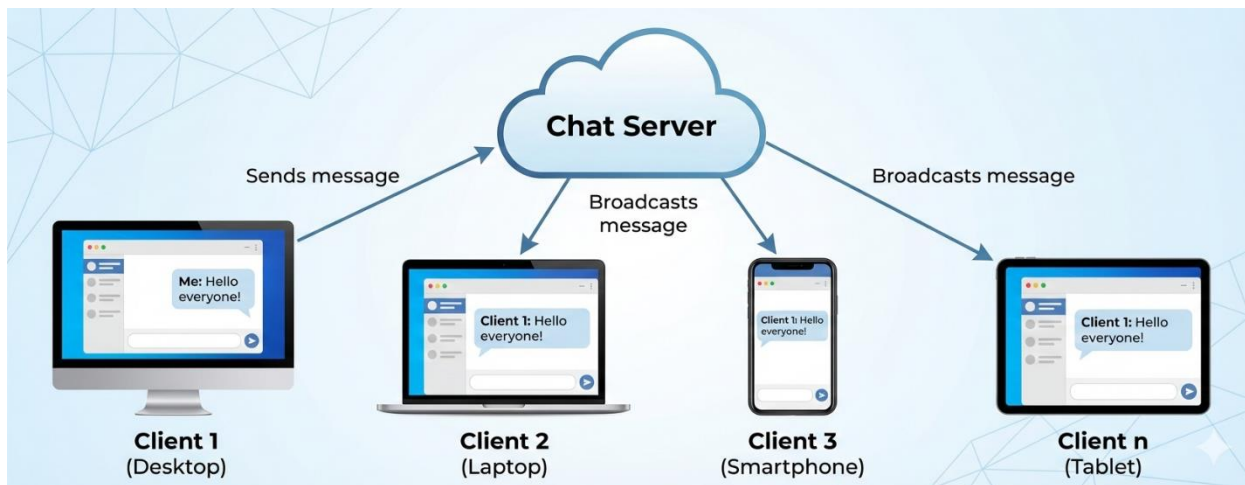
Chapter (3): Multi-Client Chat Room Systems

3.1 Introduction

In Chapter 2, we implemented a simple one-to-one client/server application.

In this chapter, we extend the concept to build a **multi-client chat room system**, where:

- Multiple clients connect to a single server.
- Messages from one client are broadcast to all connected clients.
- Each client uses a graphical user interface (GUI).
- The server handles clients using multithreading.



This chapter introduces:

- Multi-threaded servers
- Broadcast communication
- GUI-based distributed applications
- Exception handling in real systems

3.2 From One-to-One to Chat Room

One-to-One Model (Chapter 2)

- One server
- One client
- Direct communication

Chat Room Model (Chapter 3)

- One server
- Multiple clients
- Broadcast communication

Instead of replying to only one client, the server must:

1. Accept multiple client connections.
2. Store all connected clients.
3. Forward incoming messages to all clients.

3.3 Server Architecture for Chat Room

The server is divided into two classes

- `ChatServer`
- `ChatHandler`

This separation improves clarity and modularity.

3.3.1 ChatServer Class (Main Server)

The `ChatServer` is responsible for:

- Creating a `ServerSocket`
- Listening on a port
- Accepting client connections
- Creating a new thread (`ChatHandler`) for each client



3.3.2 Server Implementation

```

import java.io.*;
import java.net.*;
import java.util.*;

public class ChatServer {

    private ServerSocket serverSocket;
    private static Vector<ClientHandler> clients = new Vector<>();

    public ChatServer() {
        try {
            serverSocket = new ServerSocket(5005);
            System.out.println("Chat Server Started...");

            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("New client connected.");
                ClientHandler handler = new ClientHandler(socket);
                clients.add(handler);
                handler.start();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
  
```

```
    }

    } catch (IOException e) {
        e.printStackTrace();
    }
}

class ClientHandler extends Thread {
    private Socket socket;
    private BufferedReader reader;
    private PrintWriter writer;

    public ClientHandler(Socket socket) {
        this.socket = socket;
        try {
            reader = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            writer = new PrintWriter(
                socket.getOutputStream(), true);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```
public void run() {  
    try {  
        String message;  
        while ((message = reader.readLine()) != null) {  
            broadcast(message);  
        }  
    } catch (IOException e) {  
        System.out.println("Client disconnected.");  
    }  
}  
  
private void broadcast(String msg) {  
    for (ClientHandler client : clients) {  
        client.writer.println(msg);  
    }  
}  
}  
  
public static void main(String[] args) {  
    new ChatServer();  
}  
}
```

3.4 Multithreading in Chat Systems

Each connected client runs in a separate thread.

Why?

If one client is slow:

- It should not block other clients.

This model enables concurrency.

3.5 Client-Side GUI Application

The client:

- Uses a thread to continuously read messages
- Uses a send button to transmit messages



```
import javax.swing.*;
import java.awt.*;
import java.io.*;
import java.net.*;

public class ChatClient extends JFrame {

    private JTextArea chatArea;
    private JTextField messageField;
    private JButton sendButton;

    private Socket socket;
    private BufferedReader reader;
    private PrintWriter writer;
```

```
public ChatClient(String username) {  
  
    setTitle("Chat - " + username);  
    setSize(400, 400);  
    setLayout(new BorderLayout());  
  
    chatArea = new JTextArea();  
    chatArea.setEditable(false);  
  
    messageField = new JTextField();  
    sendButton = new JButton("Send");  
  
    add(new JScrollPane(chatArea), BorderLayout.CENTER);  
  
    JPanel bottomPanel = new JPanel(new BorderLayout());  
    bottomPanel.add(messageField, BorderLayout.CENTER);  
    bottomPanel.add(sendButton, BorderLayout.EAST);  
  
    add(bottomPanel, BorderLayout.SOUTH);  
  
    connectToServer();  
  
    sendButton.addActionListener(e -> sendMessage(username));  
  
    new Thread(this::receiveMessages).start();  
  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setVisible(true);  
}
```

```
private void connectToServer() {
    try {
        socket = new Socket("192.168.1.10", 5005);
        reader = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        writer = new PrintWriter(
            socket.getOutputStream(), true);
    } catch (IOException e) {
        JOptionPane.showMessageDialog(this,
            "Server is down. Please try later.");
        System.exit(0);
    }
}

private void sendMessage(String username) {
    String message = messageField.getText();
    writer.println(username + ": " + message);
    messageField.setText("");
}

private void receiveMessages() {
    try {
        String msg;
        while ((msg = reader.readLine()) != null) {
            chatArea.append(msg + "\n");
        }
    } catch (IOException e) {
        chatArea.append("Disconnected from server.\n");
    }
}

public static void main(String[] args) {
    String name = JOptionPane.showInputDialog("Enter your name:");
    new ChatClient(name);
}
}
```

3.6 Handling Server Down Scenario

Handling server failure by:

```
JOptionPane.showMessageDialog(this,  
    "Server is down. Please try later.");
```

This improves user experience.

3.7 Comparison: Chapter 2 vs Chapter 3

Feature	One-to-one client/server	Multi-client
Clients	1	Multiple
Threads	No	Yes
Communication	Direct	Broadcast
GUI	No	Yes
Complexity	Basic	Intermediate

3.8 Chapter Summary

In this chapter, we:

- Built a multi-client chat room server
- Used multithreading for concurrency
- Implemented GUI-based clients
- Handled exceptions properly
- Managed broadcast communication

This chapter marks the transition from basic socket programming to scalable distributed applications.

Assignment 3

Multi-Client Chat Room System

Part A – Coding Assignment

Objective

Build a multi-client chat room system using:

- Java TCP Sockets
 - Multithreading
 - GUI (Swing or JavaFX)
 - Proper exception handling
-

Task 1 – Basic Multi-Client Server

Requirements:

1. Create a server that:
 - Listens on port 5005
 - Accepts multiple clients
 - Creates a new thread per client
 - Broadcasts messages to all connected clients
 2. The server must:
 - Detect client disconnection
 - Remove disconnected client from list
 - Prevent crashing if one client fails
-

Task 2 – GUI Client Application

Requirements:

1. Design a GUI with:
 - Text area (chat history)
 - Text field (message input)
-

- Send button
 - 2. When user clicks Send:
 - Message is sent to server
 - Input field clears automatically
 - 3. Client must:
 - Continuously receive messages (reader thread)
 - Display incoming messages in text area
-

Task 3 – Enhanced Features

Add the following features:

1. Add username before each message
Example:

```
Heba: Hello everyone!
```

2. Display connection status:
 - "Connected to server"
 - "Disconnected from server"
3. Handle server-down scenario:
 - Show dialog message:

```
Server is currently unavailable.
```

4. Add timestamp to messages:

```
[10:42 PM] Ahmed: Hi
```

5. Allow "exit" command to disconnect gracefully.
-

Part B – MCQ Questions

1. In a multi-client chat server, each client should be handled using:

- A) One main thread only
 - B) Separate thread
 - C) Same input stream
 - D) Same socket
-

2. Which class allows a server to accept client connections?

- A) Socket
 - B) ServerSocket
 - C) Thread
 - D) JFrame
-

3. The purpose of broadcast() in chat systems is to:

- A) Close all connections
 - B) Send message to one client
 - C) Send message to all connected clients
 - D) Restart server
-

4. If one client blocks without threading, the server will:

- A) Continue normally
 - B) Crash immediately
 - C) Block other clients
 - D) Increase speed
-

5. Which collection is used to store connected clients?

- A) ArrayList
 - B) Vector
 - C) HashMap
 - D) File
-

6. Which Swing component is used to display multiple lines of text?

- A) JTextField
 - B) JButton
 - C) JTextArea
 - D) JLabel
-

7. The method run() belongs to:

- A) JFrame
 - B) Thread
 - C) Socket
 - D) ServerSocket
-

8. Which statement is correct about multithreading?

- A) It decreases concurrency
 - B) It allows parallel handling of clients
 - C) It removes sockets
 - D) It replaces TCP
-

9. If the server is down, the client should:

- A) Ignore the error
 - B) Freeze
 - C) Show a user-friendly message
 - D) Restart the computer
-

10. Which method reads data from socket safely?

- A) readLine() from DataInputStream
 - B) BufferedReader.readLine()
 - C) System.read()
 - D) FileReader.read()
-

11. What happens if we do not remove disconnected clients from the list?

- A) No effect
 - B) Memory leak or errors
 - C) Faster performance
 - D) Improved scalability
-

12. The thread-per-client model improves:

- A) Performance for one client
 - B) Concurrency
 - C) File storage
 - D) GUI design
-

13. A chat room server primarily uses:

- A) UDP
 - B) TCP
 - C) HTTP only
 - D) FTP
-

14. The purpose of try-with-resources is to:

- A) Increase GUI size
 - B) Automatically close resources
 - C) Create threads
 - D) Handle ports
-

15. Which architecture does a chat room follow?

- A) Peer-to-peer only
 - B) Centralized client-server
 - C) Standalone system
 - D) Embedded system
-

Chapter (4): Remote Method Invocation (RMI)

4.1 Introduction

In previous chapters, we built distributed applications using **low-level socket programming**. Although powerful, socket programming requires handling:

- Network connections manually
- Data serialization
- Thread management
- Communication protocols

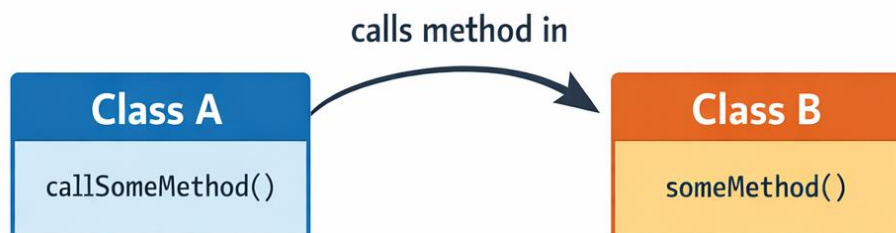
To simplify distributed programming, higher-level abstractions were developed. One such abstraction is **Remote Method Invocation (RMI)**.

4.2 From Local Method Calls to Remote Calls

Before understanding RMI, we must understand how method references work in Java.

4.2.1 Traditional Object Reference (Local Reference)

Consider two classes in the **same program**:



This program:

- Compiles successfully
- Runs normally
- Requires Class B at compile time

If `Class B` does not exist, the program **will not compile**

4.2.2 Reference to Local Interface

If Class A references an **interface**, and Class B implements that interface:



The program:

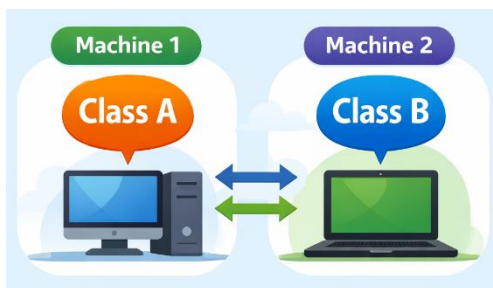
- Compiles even if `BImpl` does not exist
- Requires implementation only at runtime

This improves flexibility.

4.2.3 Reference to Remote Interface

Now imagine:

- Class A runs on Machine 1
- Class B runs on Machine 2



How can A call B's methods?

This is where **RMI** makes it possible

4.3 Remote Procedure Call (RPC)

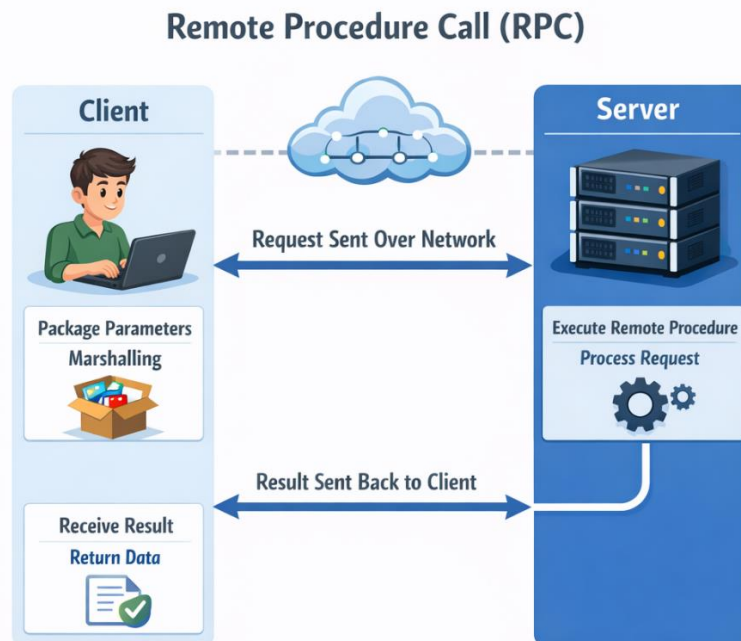
Before RMI, the concept of **Remote Procedure Call (RPC)** was introduced.

Definition

- RPC abstracts network communication to look like a normal procedure call.
- Instead of writing socket code, the programmer writes:

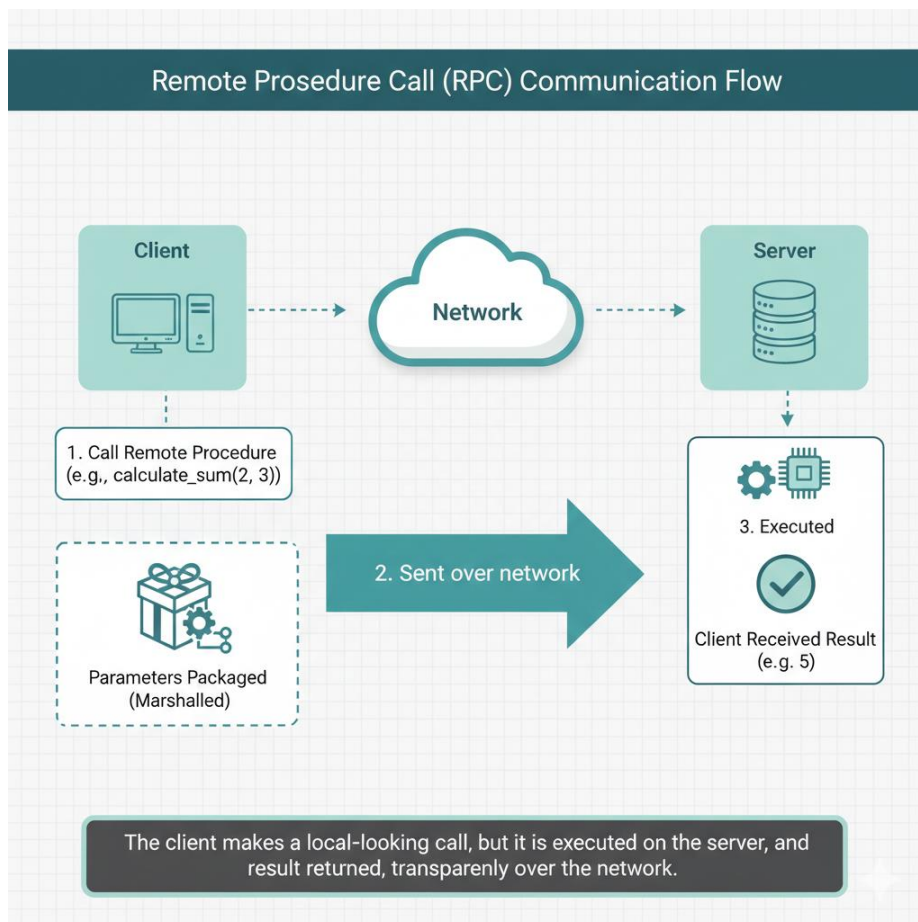
```
result = remoteObject.method(parameters);
```

- Behind the scenes:
 - Parameters are packaged (marshalled)
 - Sent over network
 - Executed remotely
 - Result is returned



- The programmer has the illusion of calling a local method.

➤ Example:

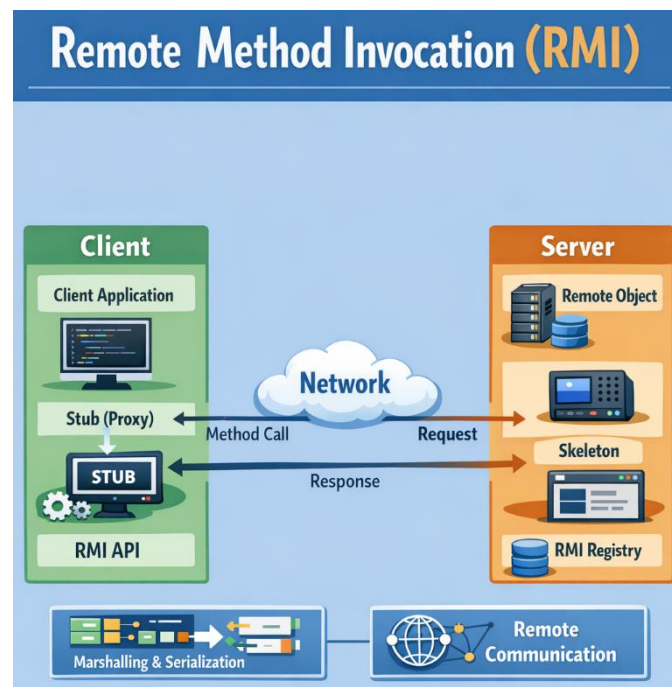


4.4 What is RMI?

Remote Method Invocation (RMI) is a Java framework for building distributed applications

It allows:

- Calling methods on remote objects
- Using Java objects directly
- Avoiding manual socket handling



4.5 RMI Characteristics

RMI:

1. Uses RPC-like communication
2. Supports object-oriented communication
3. Automatically handles serialization
4. Hides low-level networking
5. Allows calling remote methods like local methods

4.6 Components of an RMI Application

To develop an RMI application, we need

1. Remote Interface
2. Implementation Class
3. Stub and Skeleton
4. Server Application
5. Client Application

Let's explain each.

4.6.1 Remote Interface

- Defines the available remote methods
- Extends `java.rmi.Remote`
- Methods throw `RemoteException`

Example:

```
import java.rmi.Remote;  
import java.rmi.RemoteException;  
  
public interface HelloService extends Remote {  
    String sayHello(String name) throws RemoteException;  
}
```

4.6.2 Implementation Class

- Implements the remote interface
- Contains business logic
- Extends `UnicastRemoteObject`

Example:

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class HelloServiceImpl
    extends UnicastRemoteObject
    implements HelloService {

    protected HelloServiceImpl() throws RemoteException {
        super();
    }

    public String sayHello(String name)
        throws RemoteException {
        return "Hello " + name;
    }
}
```

4.6.3 Stub and Skeleton

RMI automatically generates helper classes.

Stub (Client Side)

Acts as a proxy for the remote object.

Responsibilities

- Initiates connection
- Serializes parameters (marshalling)
- Sends request
- Waits for response
- Deserializes return value (unmarshalling)

The stub hides network complexity.

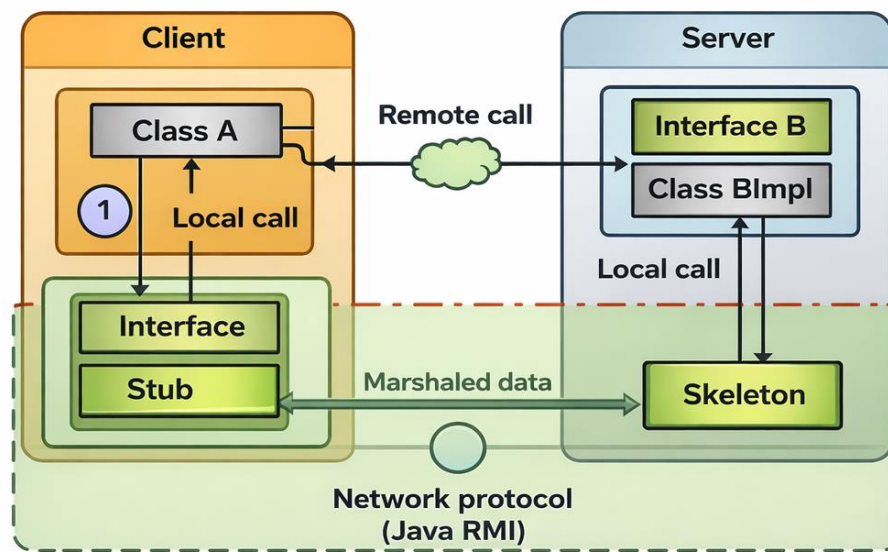
Skeleton (Server Side)

Located on server JVM.

Responsibilities

- Receives method call
- Unmarshals parameters
- Invokes real method
- Marshals result
- Sends response back

Note: In modern Java versions, skeleton is handled internally by the RMI runtime.

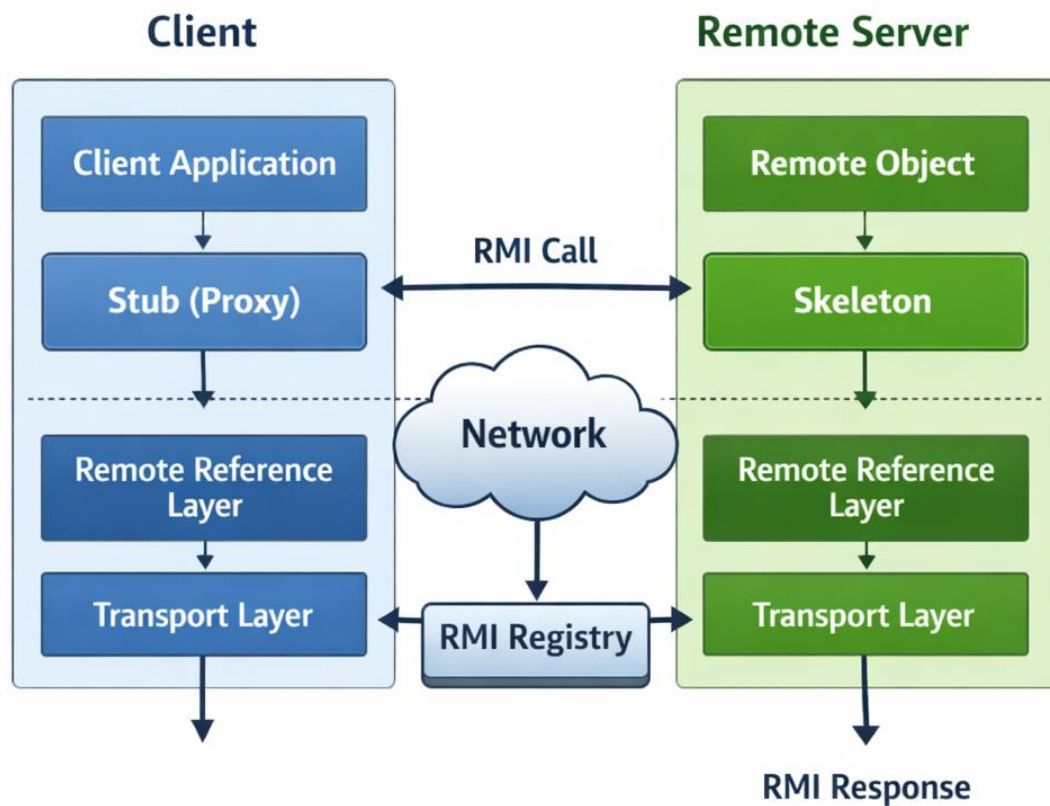


4.7 RMI Architecture

The RMI architecture consists of:

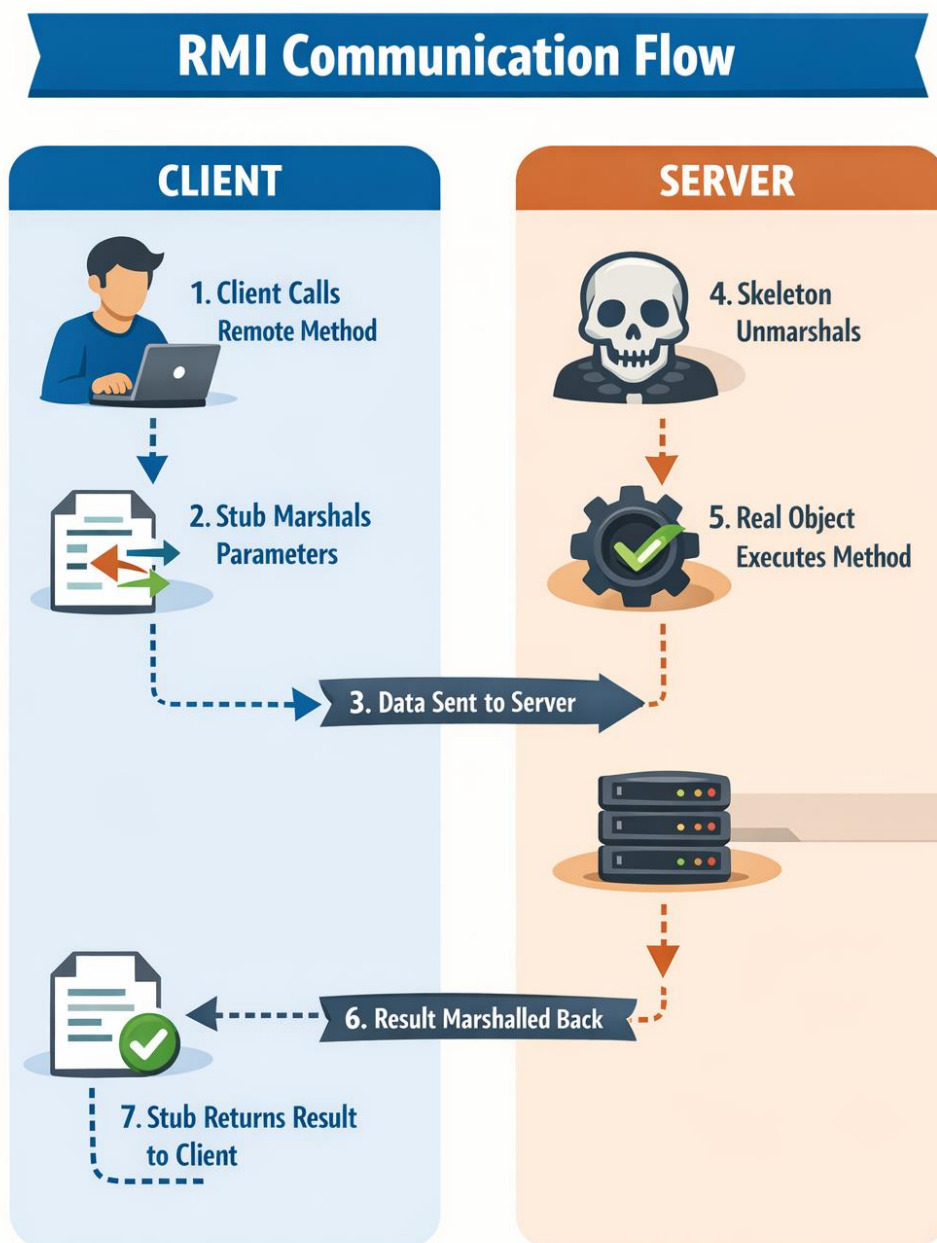
- Client Application
- Stub
- RMI Runtime
- Skeleton
- Remote Object
- Server Application

RMI Architecture



4.7.1 RMI Communication Flow

- Step 1: Client calls remote method
- Step 2: Stub marshals parameters
- Step 3: Data sent to server
- Step 4: Skeleton unmarshals
- Step 5: Real object executes method
- Step 6: Result marshalled back
- Step 7: Stub returns result to client



4.8 How RMI Works (Conceptual Overview)

1. Define Remote Interface
2. Implement the Interface
3. Remote object becomes available to clients
4. Client invokes remote method through stub
5. RMI runtime handles communication transparently

(Details of remote object registration and discovery will be explained in Chapter 5.)

4.9 Comparison: Socket vs RMI

Feature	Socket	RMI
Abstraction Level	Low	High
Serialization	Manual	Automatic
Object-Oriented	No	Yes
Ease of Use	Complex	Easier
Performance	Slightly faster	Slightly slower

4.10 Advantages of RMI

- Simplifies distributed programming
- True object-oriented communication
- Automatic serialization
- Built-in security features

4.11 Limitations of RMI

- Java-to-Java only
- Slower than raw sockets
- More complex deployment
- Firewall restrictions

4.12 Chapter Summary

In this chapter, we learned:

- The concept of RPC
- What RMI is
- How RMI abstracts network communication
- RMI architecture
- Stub and skeleton roles
- Conceptual structure of RMI-based distributed systems.

RMI represents a higher abstraction level compared to socket programming.

Assignment 4

Part A – Coding Assignment

Objective

Understand RMI architecture and implement remote method invocation logic **without manually starting or binding to the RMI registry** (registry configuration will be covered in Chapter 5).

Students will focus on:

- Remote Interface
- Implementation Class
- Understanding Stub behavior
- Exception handling
- Distributed execution model

Task 1 – Remote Interface & Implementation

Requirements

1. Create a remote interface:

```
CalculatorService
```

2. The interface must:

- Extend `java.rmi.Remote`
- Declare the following methods:

```
int add(int a, int b) throws RemoteException;  
int subtract(int a, int b) throws RemoteException;  
int multiply(int a, int b) throws RemoteException;  
int divide(int a, int b) throws RemoteException;
```

3. Implement the interface in:

```
CalculatorServiceImpl
```

4. The implementation class must:
 - Extend `UnicastRemoteObject`
 - Provide business logic for all methods
-

Task 2 – Exception Handling

Enhance your implementation by:

1. Handling division by zero properly.
2. Throwing meaningful exceptions.
3. Logging invocation attempts in server console:

```
Method add() invoked  
Method divide() invoked
```

4. Add comments explaining where:
 - Marshalling occurs
 - Unmarshalling occurs
 - Stub participates in communication

(Conceptual explanation required in code comments.)

Task 3 – Conceptual Analysis

Answer the following inside a PDF report:

1. What is the role of the Stub?
 2. What is the role of the Skeleton?
 3. What is marshalling?
 4. How is RMI different from socket programming?
 5. Why must remote methods throw `RemoteException`?
-

Part B – MCQ Questions

1. RMI stands for:

- A) Remote Machine Interaction
 - B) Remote Method Invocation
 - C) Remote Message Interface
 - D) Remote Module Integration
-

2. A remote interface must extend:

- A) Serializable
 - B) Runnable
 - C) Remote
 - D) Cloneable
-

3. All remote methods must declare:

- A) IOException
 - B) RemoteException
 - C) RuntimeException
 - D) SQLException
-

4. In RMI, marshalling refers to:

- A) Encrypting data
 - B) Converting objects into bytes for transmission
 - C) Opening sockets manually
 - D) Closing remote connection
-

5. The Stub is responsible for:

- A) Executing business logic
 - B) Acting as a proxy for the remote object
-

- C) Managing GUI
 - D) Storing database records
-

6. The Skeleton is responsible for:

- A) Handling GUI
 - B) Receiving method calls on server side
 - C) Managing threads
 - D) Encrypting data
-

7. RMI provides abstraction over:

- A) GUI programming
 - B) Database systems
 - C) Low-level socket communication
 - D) Operating system
-

8. RMI mainly supports:

- A) Object-oriented distributed programming
 - B) Only primitive data transfer
 - C) File compression
 - D) Hardware access
-

9. Remote objects typically extend:

- A) Thread
 - B) JFrame
 - C) UnicastRemoteObject
 - D) File
-

10. Compared to sockets, RMI is:

- A) Lower-level
 - B) Higher-level abstraction
-

- C) Hardware-dependent
 - D) Protocol-free
-

11. Why must RemoteException be declared?

- A) Because network failures may occur
 - B) Because of arithmetic operations
 - C) Because GUI requires it
 - D) Because of memory limits
-

12. Which component hides network complexity from the client?

- A) Skeleton
 - B) Stub
 - C) Thread
 - D) Socket
-

13. RMI uses which underlying protocol?

- A) UDP
 - B) TCP
 - C) HTTP
 - D) FTP
-

14. Unmarshalling refers to:

- A) Converting bytes back into objects
 - B) Encrypting data
 - C) Removing exceptions
 - D) Closing streams
-

15. RMI enables:

- A) Calling remote methods like local methods
 - B) Eliminating exceptions
-

- C) Replacing Java
- D) Avoiding object-oriented design

Chapter (5): RMI Registry and Deployment

5.1 Introduction

In Chapter 4, we studied the architecture of Remote Method Invocation (RMI), including:

- Remote interfaces
- Implementation classes
- Stub and skeleton
- Marshalling and unmarshalling
- RMI communication flow

However, one essential question remains:

How does a client locate a remote object on the network?

The answer is: **RMI Registry**.

This chapter explains:

- The registry problem
- The RMI Registry
- Service binding and lookup
- Modern Hello RMI example (Java 8+ / 11 / 17 / 21)
- Listing services using `registry.list()`
- Serialization in RMI
- Compilation and execution steps

5.2 The Registry Problem

In early distributed systems, clients needed:

- Exact server IP address
- Exact port number
- Direct reference to remote object

If the server location changed, every client had to be modified manually.

This caused:

- Tight coupling
- Deployment difficulties
- Maintenance overhead
- Scalability limitations

A naming service was required.

5.3 The RMI Registry

The **RMI Registry** is a naming service that:

- Maps a service name (String) to a remote object
- Allows clients to discover remote services
- Runs by default on port **1099**

It follows the **Broker Architecture Pattern**.

How the Registry Can Run

The registry can:

1. Run as a standalone process using:
`rmiregistry`
2. Be started programmatically inside the server:
`LocateRegistry.createRegistry(1099);`

Both approaches are valid.



5.4 How RMI Works Using the Registry

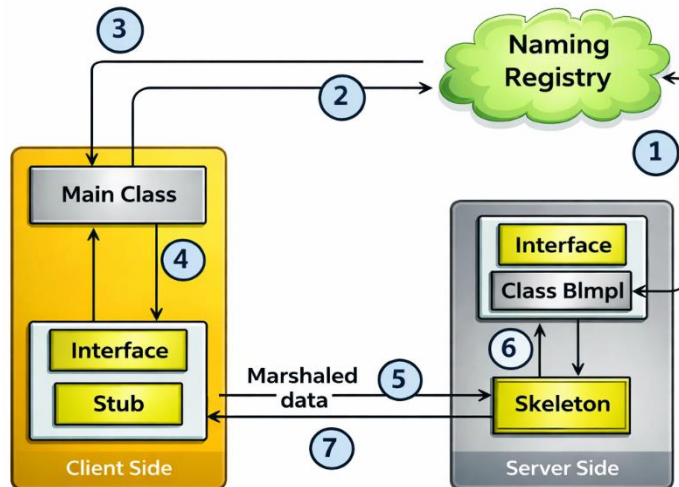
The complete process is:

Server Side

1. Start the RMI registry.
2. Create remote object implementation.
3. Bind the object to the registry with a name.

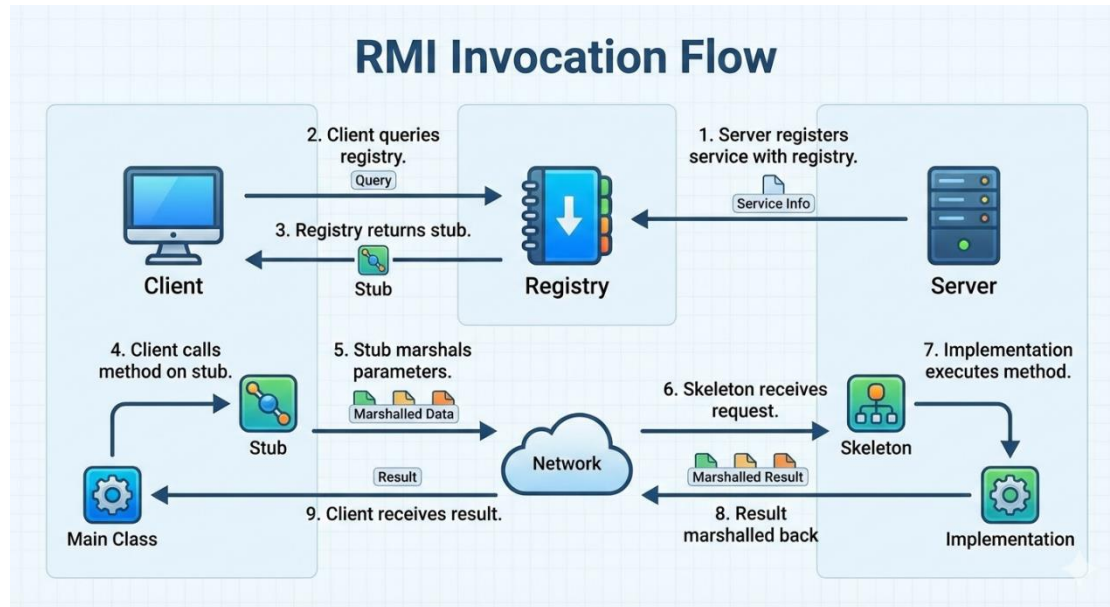
Client Side

1. Connect to registry.
2. Look up service by name.
3. Receive stub (proxy).
4. Invoke remote methods.



5.5 Detailed RMI Invocation Flow

1. Server registers service with registry.
2. Client queries registry.
3. Registry returns stub.
4. Client calls method on stub.
5. Stub marshals parameters.
6. Skeleton receives request.
7. Implementation executes method.
8. Result marshalled back.
9. Client receives result



5.6 Complete Hello RMI Example

This example uses modern Java. No `rmic` command is required. Stubs are generated dynamically at runtime.

1. Remote Interface `HelloService.java`

```

import java.rmi.Remote;
import java.rmi.RemoteException;

/**
 * Remote Interface:
 * Defines the methods that can be invoked remotely.
 */
public interface HelloService extends Remote {

    // Every remote method must declare RemoteException
    String sayHello(String name) throws RemoteException;
}
  
```

2. Implementation Class `HelloServiceImpl.java`

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

/**
 * Remote Object Implementation:
 * - Implements the remote interface
 * - Extends UnicastRemoteObject to automatically export the object
 */
public class HelloServiceImpl extends UnicastRemoteObject implements HelloService {

    // Constructor must throw RemoteException
    public HelloServiceImpl() throws RemoteException {
        super(); // exports the object on an anonymous port
    }

    @Override
    public String sayHello(String name) throws RemoteException {

        if (name == null || name.isBlank()) {
            name = "Anonymous";
        }

        return "Hello " + name + " 🤖 (from RMI Server)";
    }
}
```

3. Server Application `RMIServer.java`

Responsible for:

- Starting registry
- Binding remote object
- Keeping service alive

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

/**
 * RMI Server Application:
 * - Starts the registry
 * - Creates the remote object
 * - Binds it using a service name
 */
public class RMIServer {

    private static final int PORT = 1099;

    public static void main(String[] args) {

        try {
            // Start registry programmatically
            Registry registry = LocateRegistry.createRegistry(PORT);

            // Create remote service object
            HelloService helloService = new HelloServiceImpl();

            // Bind service with registry
            registry.rebind("HelloService", helloService);
            registry.rebind("GreetingService", helloService);

            System.out.println("RMI Server Running...");
            System.out.println("Services Registered:");
            System.out.println(" - HelloService");
            System.out.println(" - GreetingService");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

4. Client Application `RMIClient.java`

Responsible for:

- Locating registry
- Looking up service
- Invoking remote methods

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

/**
 * RMI Client Application:
 * - Connects to registry
 * - Lists available services
 * - Looks up remote object
 * - Invokes remote method
 */
public class RMIClient {

    private static final int PORT = 1099;

    public static void main(String[] args) {
```

```
        try {
            String host = "localhost";

            // Allow passing server IP as argument
            if (args.length >= 1) {
                host = args[0];
            }
        }
```

```
// Connect to registry
Registry registry = LocateRegistry.getRegistry(host, PORT);

System.out.println("Connected to Registry at " + host);

// List all registered services
System.out.println("\nAvailable Services:");
String[] services = registry.list();

for (String serviceName : services) {
    System.out.println(" - " + serviceName);
}
```

```
// Lookup specific service
HelloService service =
    (HelloService) registry.lookup("HelloService");

// Invoke remote method
String reply = service.sayHello("Heba");

System.out.println("\nServer Response:");
System.out.println(reply);

} catch (Exception e) {
    e.printStackTrace();
}
}
```

5.7 Service Discovery Using registry.list()

The registry provides:

```
registry.list();
```


This method:

- Returns `String[]`
- Contains names of all registered services

Example output:

```
Available Services:  
- HelloService  
- GreetingService
```

This demonstrates dynamic service discovery.

5.8 Compile and Run Hello RMI Example

How to Compile & Run

Open terminal **in the folder that contains the 4 files**:

Step 1 – Compile All files

```
javac *.java
```

Step 2 – Run Server

```
java RMIServer
```

Step 3 – Run Client

```
java RMIClient
```

Or:

```
java RMIClient 192.168.1.10
```

5.9 Serialization in RMI

What is Serialization?

Serialization converts objects into byte streams.

RMI uses serialization to:

- Transfer parameters
- Return objects

Objects passed by value must implement:

```
import java.io.Serializable;
```

Example:

```
public class Employee implements Serializable {

    private int id;
    private String name;
    private double salary;

    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }
}
```



5.10 Advantages of RMI

- Location transparency
- Dynamic service discovery
- Decouples client and server
- Simplifies distributed programming

5.11 Limitations of RMI

- Java-to-Java only
 - Firewall configuration required
 - Deployment complexity
-

5.12 Chapter Summary

In this chapter, we:

- Introduced the RMI Registry
- Solved the naming problem
- Implemented modern Hello RMI example
- Used `registry.list()` for service discovery
- Explained serialization
- Covered execution steps

This completes the practical RMI deployment lifecycle.

Assignment 5

Part A – Coding Assignment

Objective

Develop complete RMI-based distributed applications using:

- RMI Registry
- Remote Interface
- Implementation Class
- Server Application
- Client Application
- Proper exception handling
- Serialization (where required)

Task 1 – Hello RMI Application

Requirements

1. Create a remote interface:

```
HelloService
```

2. Define the method:

```
String sayHello(String name) throws RemoteException;
```

3. Implement the interface in:

```
HelloServiceImpl
```

4. Create a server application that:

- Starts the RMI registry
- Registers the remote object using:

```
registry.rebind("HelloService", service);
```

- Displays confirmation message:

```
Service Registered Successfully
```

5. Create a client application that:

- Connects to the registry
- Looks up "HelloService"
- Invokes `sayHello()`
- Prints the result

Task 2 – Distributed Service Directory System

Objective

Build an RMI-based **Service Directory System** that demonstrates proper use of the RMI Registry for service discovery.

Problem Description

In this task, you will create a distributed system where:

- The server registers a remote service in the RMI registry.
- The client retrieves available services from the registry.
- The client invokes remote methods dynamically.

This task emphasizes **registry binding, lookup, and service discovery**.

Requirements

1. Remote Interface

Create an interface:

`DirectoryService`

It must define the following methods:

`String getServiceDescription(String serviceName) throws RemoteException;`

`String[] listAvailableServices() throws RemoteException;`

2. Implementation Class

Create:

`DirectoryServiceImpl`

The implementation must:

- Extend `UnicastRemoteObject`
- Maintain a simple in-memory mapping such as:

Service Name	Description
"MathService"	Performs arithmetic operations
"WeatherService"	Provides weather information
"EmployeeService"	Manages employee records

3. Server Application

The server must:

- Start the RMI registry on port 1099
- Create the service implementation object
- Bind the service using:

```
registry.rebind("DirectoryService", service);
```

- Print:

```
DirectoryService Registered Successfully
```

4. Client Application

The client must:

1. Connect to the registry.
2. Retrieve the remote object using:

```
registry.lookup("DirectoryService");
```

3. Call:

```
listAvailableServices()
```

4. Allow user to enter a service name.
5. Call:

```
getServiceDescription(serviceName)
```

6. Display the result on console.
-

Additional Requirements

- Handle `NotBoundException`
 - Handle `RemoteException`
 - Handle incorrect service names
 - Display a clear message if registry is not running
-

Task 3 – Remote Database Application

Create a simple RMI-based Employee Management System.

Server Side Must Provide:

- `addEmployee(Employee e)`
- `updateEmployee(Employee e)`
- `deleteEmployee(int id)`
- `getEmployee(int id)`
- `getAllEmployees()`

Employee Class Requirements:

- Must implement `Serializable`
- Contain:
 - `id`
 - `name`
 - `salary`

Client Application Must:

- Retrieve and display:
 - Single employee
 - All employees
- Demonstrate CRUD operations remotely

Part B – MCQ Questions

1. The RMI Registry acts as:

- A) Database
 - B) Naming service
 - C) Thread manager
 - D) GUI controller
-

2. The default RMI registry port is:

- A) 5005
 - B) 8080
 - C) 1099
 - D) 21
-

3. The method used to bind a service is:

- A) add()
 - B) register()
 - C) rebind()
 - D) export()
-

4. To retrieve a remote object, the client uses:

- A) bind()
 - B) rebind()
 - C) lookup()
 - D) connect()
-

5. The registry provides:

- A) Business logic
 - B) Network cables
 - C) Mapping between names and remote objects
 - D) Database storage
-

6. Which command starts the registry manually?

- A) java registry
 - B) rmiregistry
 - C) start rmi
 - D) rmi start
-

7. If the service name is incorrect, the client throws:

- A) NullPointerException
 - B) NotBoundException
 - C) ClassCastException
 - D) ArithmeticException
-

8. Objects transferred by value must implement:

- A) Runnable
 - B) Remote
 - C) Serializable
 - D) Cloneable
-

9. In RMI, remote objects are passed as:

- A) Serialized copies
 - B) Stubs
 - C) Files
 - D) Threads
-

10. The registry can run:

- A) Only on client
 - B) Only on server
 - C) On server or separate machine
 - D) Only inside JVM
-

11. Serialization converts:

- A) Methods into threads
 - B) Objects into bytes
 - C) Exceptions into logs
 - D) Ports into IP
-

12. In RMI, binding allows:

- A) Direct socket communication
 - B) Remote object registration
 - C) Database insertion
 - D) Thread creation
-

13. The client must know:

- A) Implementation class
 - B) Remote interface
 - C) Server source code
 - D) Stub code manually
-

14. Compared to sockets, RMI registry:

- A) Removes naming problems
 - B) Increases manual configuration
 - C) Disables remote calls
 - D) Replaces Java
-

15. The registry provides which architecture style?

- A) Peer-to-peer
- B) Broker architecture
- C) Embedded architecture
- D) Standalone architecture

Chapter (6): RMI with Callback

6.1 Introduction

In previous chapters, we studied:

- Basic RMI architecture
- RMI Registry
- Remote method invocation from client to server

So far, communication has been **unidirectional**:

Client calls methods on server.

But what if the server needs to:

- Notify clients?
- Send asynchronous updates?
- Push messages?
- Deliver chat messages?

This requires **Callback Mechanism**.

6.2 What is Callback?

A **callback** is a mechanism that allows:

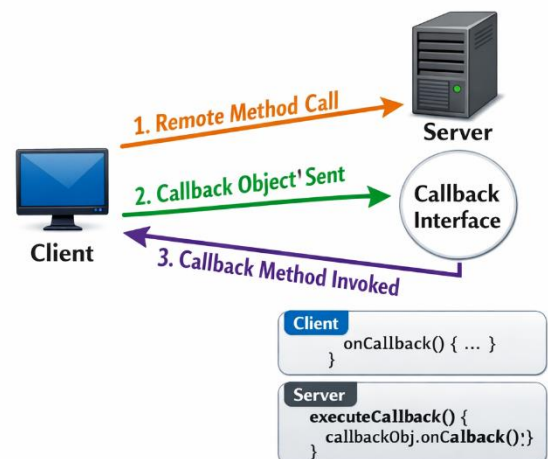
The server to invoke methods on the client.

In RMI Callback:

- The client becomes both:
 - A client (calls server)
 - A server (receives remote calls)

This enables **bidirectional communication**.

RMI with Callback

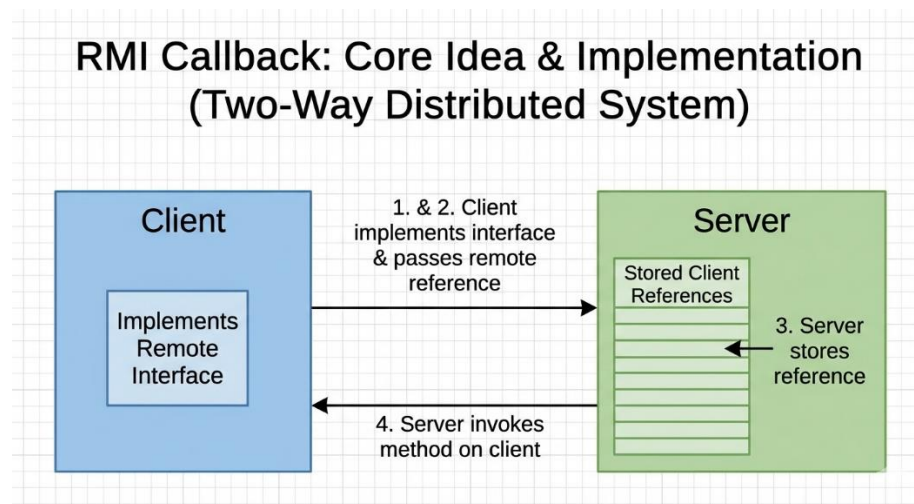


6.3 Core Idea of RMI Callback

The callback is implemented by:

1. Making the client implement a remote interface.
2. Passing the client's remote reference to the server.
3. The server storing that reference.
4. The server invoking methods on the client.

This creates a two-way distributed system.



6.4 How Callback Works

Step 1 – Define a Client Remote Interface

At the client side, define a remote interface that includes:

- Methods the server is allowed to call.

Example:

```
import java.rmi.Remote;  
import java.rmi.RemoteException;  
  
public interface ClientCallback extends Remote {  
  
    void receiveMessage(String message) throws RemoteException;  
}
```

This restricts the server to only invoke specific methods.

6.5 Client Implementation

The client must:

- Implement `ClientCallback`
- Extend `UnicastRemoteObject`

```
import java.rmi.RemoteException;  
import java.rmi.server.UnicastRemoteObject;  
  
public class ClientCallbackImpl  
    extends UnicastRemoteObject  
    implements ClientCallback {  
  
    public ClientCallbackImpl() throws RemoteException {  
        super();  
    }  
  
    @Override  
    public void receiveMessage(String message)  
        throws RemoteException {  
        System.out.println("New message: " + message);  
    }  
}
```

Important:

- This object is NOT registered in the registry.
- It is passed directly to the server.

6.6 Server Remote Interface (With Registration)

The server must define a registration method that accepts:

- The client's remote reference.

Example:

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface ChatServer extends Remote {

    void registerClient(ClientCallback client)
        throws RemoteException;

    void unregisterClient(ClientCallback client)
        throws RemoteException;

    void sendMessage(String message)
        throws RemoteException;
}
```

6.7 Server Implementation

The server stores connected clients.

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
import java.util.ArrayList;
import java.util.List;

public class ChatServerImpl
    extends UnicastRemoteObject
    implements ChatServer {

    private List<ClientCallback> clients;

    public ChatServerImpl() throws RemoteException {
        super();
        clients = new ArrayList<>();
    }
```

```
@Override
public void registerClient(ClientCallback client)
    throws RemoteException {
    clients.add(client);
    System.out.println("Client registered.");
}

@Override
public void unregisterClient(ClientCallback client)
    throws RemoteException {
    clients.remove(client);
    System.out.println("Client unregistered.");
}
```



```
@Override
public void sendMessage(String message)
    throws RemoteException {

    for (ClientCallback client : clients) {
        client.receiveMessage(message);
    }
}
}
```

6.8 Chatting Example Architecture

Server Exposes:

- registerClient()
- unregisterClient()
- sendMessage()

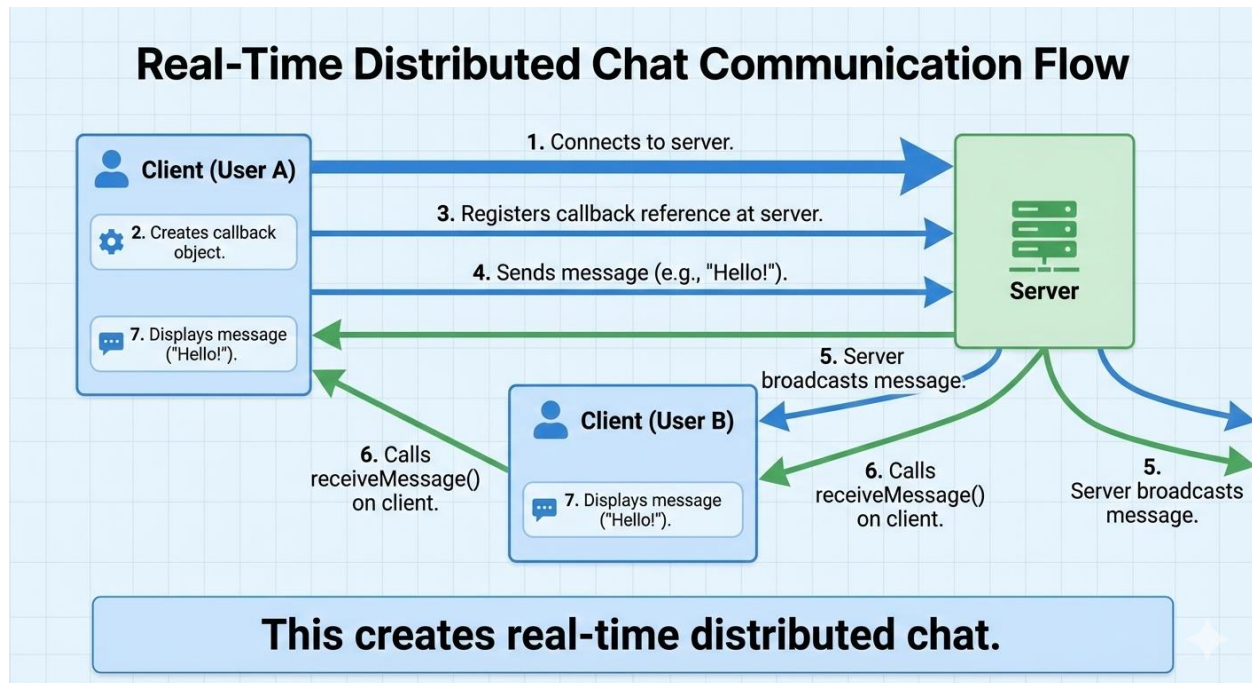
Client Exposes:

- receiveMessage()

6.9 Communication Flow

1. Client connects to server.
2. Client creates callback object.
3. Client registers callback reference at server.
4. Client sends message.
5. Server broadcasts message.
6. Server calls receiveMessage() on each client.
7. Clients display message.

This creates **real-time distributed chat**.



6.10 Why Callback Is Important

Callback enables:

- Event-driven distributed systems
- Push-based updates
- Real-time chat systems
- Notification systems
- Collaborative applications

It simulates a **bidirectional communication channel** over RMI.

6.11 Chapter Summary

In this chapter, we:

- Introduced RMI Callback
- Implemented client-side remote interface
- Enabled server-to-client invocation
- Built chat architecture

- Designed GUI-based distributed system
- Learned bidirectional RMI communication

This chapter completes advanced RMI communication concepts.

6.12 Lab Exercise

Objective

Implement a complete GUI-based RMI Chat Application using Callback.

Requirements

1. Complete Server

- Implement ChatServerImpl
 - Start registry
 - Bind ChatServer
 - Maintain list of connected clients
-

2. Complete Client

- Implement ClientCallbackImpl
 - Connect to registry
 - Register at server
 - Send messages to server
-

3. GUI Client Application

Create a GUI containing:

- Text Area (chat history)
- Text Field (message input)
- Send Button

Use:

- Swing (JFrame)
 - Threads when sending messages
 - Avoid blocking UI
-

4. Advanced Requirement

- Display username with each message.
 - Allow client logout.
 - Handle disconnected clients.
-

6.13 MCQ Questions

1. RMI Callback enables:
A) Client-to-client communication only
B) Server to invoke client methods
C) UDP communication
D) Database storage
2. In RMI Callback, the client must:
A) Only call server
B) Implement a remote interface
C) Start registry
D) Use UDP
3. The client callback object:
A) Must be registered in registry
B) Is passed directly to server
C) Is stored in database
D) Uses HTTP
4. The server stores:
A) IP addresses only

- B) Client remote references
 - C) Files
 - D) Ports
5. Callback creates:
- A) Unidirectional communication
 - B) Bidirectional communication
 - C) Stateless system
 - D) Local-only communication
6. The method `receiveMessage()` belongs to:
- A) Server
 - B) Client
 - C) Registry
 - D) Database
7. To avoid blocking GUI, we should:
- A) Remove threads
 - B) Use threads
 - C) Stop registry
 - D) Close sockets
8. Callback is useful for:
- A) Real-time updates
 - B) File compression
 - C) Hardware control
 - D) OS installation
9. In callback, the client acts as:
- A) Only client
 - B) Only server
 - C) Both client and server
 - D) Database

10. The server must provide:
- A) Registration method
 - B) Only sendMessage
 - C) Only lookup
 - D) Only registry

Practical project

Distributed Systems Project - Final Documentation Guide

Project Overview

Choose any real-world system that uses distributed computing (like Netflix, WhatsApp, online banking, cloud storage, etc.) and explain how it works as a distributed system. Balance conceptual understanding with technical insight - explain both what the system does and how it achieves distributed computing goals.

Required Sections for Your Report

1. Introduction

Write a brief overview of your chosen system. What is it? Who uses it? Why is it important? Include basic technical context about the system's scale and complexity.

2. Project Goals

Explain what you aim to demonstrate through this project. What specific distributed system concepts and technical implementations will you explore?

3. System Architecture Overview

Describe the overall structure of your chosen system. How are the different parts connected? Include:

- Network topology and communication patterns
- Data flow between components
- Key architectural patterns used (microservices, client-server, peer-to-peer, etc.)
- Simple diagrams showing technical architecture

4. System Components

List and explain the main parts of your system, including technical details:

- Servers, databases, user interfaces, APIs, load balancers
- What protocols each component uses for communication
- Data structures and storage mechanisms
- How components interact technically

5. Key Features and Functionality

Describe the main features your system offers to users. For each feature, explain:

- The technical implementation approach
- Which distributed computing principles enable the feature
- Algorithms or protocols that make it work across multiple machines

6. Why This Qualifies as a Distributed System

Prove that your chosen system is actually distributed by explaining how it technically:

- Uses multiple computers working together (include network protocols)
- Shares resources across a network (describe resource sharing mechanisms)
- Operates without a single point of control (explain coordination algorithms)
- Implements distributed computing principles in practice

7. Distributed System Challenges (Select at least 7)

Choose the challenges that apply to your system and explain both the conceptual challenge and technical solutions:

****Scalability:**** How does the system handle increasing load? Include specific scaling techniques, algorithms, or architectural patterns used.

****Security:**** What security protocols, encryption methods, and authentication mechanisms protect the distributed system?

****Transparency:**** What technical mechanisms hide the distributed nature from users? Include specific protocols or middleware.

****Openness:**** What standards, APIs, and protocols enable interoperability with different systems?

****Consistency:**** What consistency models and algorithms (like consensus protocols) ensure data synchronization across distributed nodes?

****Load Balancing:**** What load balancing algorithms and techniques distribute work? How do they technically operate?

****Failure Handling:**** What fault tolerance mechanisms, redundancy strategies, and recovery protocols handle system failures?

****Concurrency:**** How does the system manage concurrent operations? Include synchronization mechanisms and concurrency control methods.

****Heterogeneity:**** What technical approaches allow different hardware/software to work together? Include relevant protocols or standards.

****Quality of Service:**** What technical mechanisms ensure performance? Include monitoring systems, resource allocation algorithms, or service level agreements.

8. Conclusion

Summarize what you learned about distributed systems through studying your chosen example. Connect the technical implementations to distributed systems theory and principles.

Research and Analysis Guidelines:

- Choose a system you can research thoroughly with available technical documentation
- Include relevant protocols, algorithms, and data structures in your explanations
- Use technical diagrams and system architecture illustrations
- Cite technical papers, documentation, or reliable sources about the system's implementation
- Explain how technical solutions address theoretical distributed systems challenges
- Balance theoretical concepts with real-world technical implementations
- Aim for 8-15 pages with technical depth

